矩阵阶梯形与 QR 分解实验技术报告

基于 main.py 与 matlibrary.py 的实现与验证

李谨硕, 525030910098

2026 年 5 月 17 日

摘要

本报告围绕三个线性代数程序实验展开：随机生成 10×8 整数矩阵并化为行阶梯形矩阵；使用 Householder 变换实现指定矩阵的 QR 分解；使用 Givens 旋转实现指定矩阵的 QR 分解。程序主体由 main.py 调度，核心矩阵类与算法由 matlibrary.py 提供。报告分为三个小节，每个小节均包含问题及算法描述、程序验证结果、反思总结。附录中给出 main.py 与 matlibrary.py 的完整代码。

目录

1 实验背景与程序结构

本实验的目标是用程序实现常见矩阵分解与矩阵化简算法，并用数值残差、正交性误差以及第三方库结果进行验证。`matlibrary.py` 以一个统一的 `Matrix` 类作为核心，支持 `fraction` 与 `float` 两种数值模式，并实现了高斯消元、矩阵乘法、转置、秩、QR 分解等方法。`main.py` 负责构造三个实验矩阵、调用算法接口、打印分解结果并用 SymPy 进行对照验证。

表 1: 程序文件职责

文件	职责说明
`main.py`	生成实验输入矩阵，分别调用高斯消元、Householder QR 与 Givens QR，并输出验证指标。
`matlibrary.py`	定义 `Matrix` 类，实现矩阵存储、基础运算、高斯消元、QR 分解及相关辅助函数。
`README.md` / `README_cn.md` / `README_en.md`	说明项目定位、接口列表、算法适用范围与使用示例。

程序中使用的主要误差指标如下。若 $A = QR$ 为 QR 分解，则重构误差为

$$e_{\text{res}} = \|A - QR\|_F,$$

正交性误差为

$$e_{\text{orth}} = \|Q^T Q - I\|_F.$$

当两个误差均接近机器精度时，可以认为程序分解结果在数值意义上是正确的。

2 任务一：随机矩阵的行阶梯形化

2.1 问题及算法描述

任务要求随机生成一个 10×8 阶整数矩阵,并将其化为阶梯形矩阵。本程序使用 `random.Random(0)` 固定随机种子，从而保证实验可重复。矩阵元素由区间 $[-9, 9]$ 内的整数随机生成。

本任务采用带部分主元选取的高斯消元算法。设当前主元位于第 r 行、第 c 列，算法先在第 r 行及其下方寻找绝对值最大的非零元素作为主元，以减少除以过小数值带来的误差；随后对下方各行进行

$$R_i \leftarrow R_i - \frac{a_{ic}}{a_{rc}} R_r, \qquad i = r+1, \ldots, m.$$

当当前列无法找到非零主元时跳过该列。最终得到上阶梯形矩阵，并记录行交换次数。

2.2 程序验证结果

随机生成的矩阵为

$$A = \begin{bmatrix} 3 & 4 & -8 & -1 & 7 & 6 & 3 & 0 \\ 6 & 2 & 9 & -3 & 7 & -5 & 0 & -5 \\ -6 & -1 & 8 & -5 & 0 & -6 & -7 & 1 \\ 6 & 8 & -6 & 2 & 4 & 1 & -3 & 8 \\ 6 & 5 & 7 & -1 & -8 & 8 & -9 & -7 \\ 3 & -9 & 6 & 1 & -2 & 1 & -7 & -3 \\ 9 & -2 & -2 & -5 & 8 & 5 & -7 & -7 \\ 1 & 7 & 6 & -6 & 0 & 8 & 0 & -6 \\ 8 & 1 & 8 & -3 & 8 & 9 & 0 & 5 \\ -7 & 3 & 1 & 9 & -2 & 0 & -4 & -3 \end{bmatrix}.$$

带主元高斯消元输出的行阶梯形矩阵为

$$\mathrm{REF}(A) = \begin{bmatrix} 9.0000 & -2.0000 & -2.0000 & -5.0000 & 8.0000 & 5.0000 & -7.0000 & -7.0000 \\ 0 & 9.3333 & -4.6667 & 5.3333 & -1.3333 & -2.3333 & 1.6667 & 12.6667 \\ 0 & 0 & 12.0000 & -1.5714 & 2.1429 & -7.5000 & 4.0714 & -4.8571 \\ 0 & 0 & 0 & -8.2837 & -1.6131 & 15.3958 & -3.8482 & -11.0437 \\ 0 & 0 & 0 & 0 & -14.5250 & 13.8468 & -9.4684 & -6.5674 \\ 0 & 0 & 0 & 0 & 0 & 15.7538 & -14.0806 & -17.6896 \\ 0 & 0 & 0 & 0 & 0 & 0 & -19.4138 & 0.4436 \\ 0 & 0 & 0 & 0 & 0 & 0 & 0 & 26.4275 \\ 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 \end{bmatrix}.$$

验证结果如下：

表 2: 任务一验证结果

项目	结果
行交换次数	7
程序计算秩	8
SymPy 对照秩	8
结论	阶梯形结果与秩验证一致

2.3 反思总结

本任务中，带部分主元的高斯消元比直接消元更稳健，尤其是在随机矩阵中可能出现较小主元时，主元选取可以减少数值误差。由于矩阵为 10×8，最大秩不超过 8，本次程序

与 SymPy 均得到秩 8，说明该随机矩阵为列满秩。需要注意的是，不同软件或不同主元策略得到的阶梯形矩阵并不唯一，因此验证时不应只比较每个元素是否完全相同，而应重点比较秩、零结构以及是否可由初等行变换得到。

3 任务二：Householder 变换 QR 分解

3.1 问题及算法描述

任务要求对矩阵

$$A = \begin{bmatrix} 3 & 14 & 9 \\ 6 & 43 & 3 \\ 6 & 22 & 15 \end{bmatrix}$$

进行 QR 分解，即求正交矩阵 Q 和上三角矩阵 R，使得

$$A = QR, \qquad Q^TQ = I.$$

Householder 变换的核心思想是用反射矩阵逐列消去主元下方元素。对当前列向量 x，构造单位向量

$$v = \frac{x + \operatorname{sign}(x_1)\|x\|_2 e_1}{\|x + \operatorname{sign}(x_1)\|x\|_2 e_1\|_2},$$

再构造反射矩阵

$$H = I - 2vv^T.$$

不断将 H 作用于当前子矩阵，即可把矩阵化为上三角矩阵 R；所有反射矩阵的累积给出正交矩阵 Q。

3.2 程序验证结果

程序使用 `A2.qr_decomposition(method="householder")` 得到

$$Q = \begin{bmatrix} -0.3333 & 0.1333 & 0.9333 \\ -0.6667 & -0.7333 & -0.1333 \\ -0.6667 & 0.6667 & -0.3333 \end{bmatrix}, \qquad R = \begin{bmatrix} -9.0000 & -48.0000 & -15.0000 \\ 0 & -15.0000 & 9.0000 \\ 0 & 0 & 3.0000 \end{bmatrix}.$$

验证误差为

表 3: 任务二验证结果

项目	数值
重构误差 $\|A - QR\|_F$	9.503924×10^{-15}
正交性误差 $\|Q^TQ - I\|_F$	4.191000×10^{-16}
SymPy 重构误差	0.000000×10^{0}

可以看到，重构误差和正交性误差均接近浮点机器精度，说明 Householder QR 分解结果正确。SymPy 输出的 Q 和 R 与本程序存在若干列符号差异，这是 QR 分解中常见的非唯一性：若同时改变 Q 的某一列与 R 的对应行符号，乘积 QR 不变。

3.3 反思总结

Householder 方法通过正交反射实现消元，理论上不会放大向量范数，因此通常比经典 Gram-Schmidt 方法具有更好的数值稳定性。本次结果中，R 的下三角元素被成功消为零，且 Q 的正交性误差在 10^{-16} 量级，体现了该方法在小规模稠密矩阵 QR 分解中的可靠性。

4 任务三：Givens 旋转 QR 分解

4.1 问题及算法描述

任务要求对矩阵

$$A = \begin{bmatrix} 2 & 2 & 1 \\ 0 & 2 & 2 \\ 2 & 1 & 2 \end{bmatrix}$$

使用 Givens 变换进行 QR 分解。

Givens 旋转通过二维平面旋转逐个消去指定元素。若希望消去向量 $[a, b]^T$ 中的 b，令

$$r = \sqrt{a^2 + b^2}, \qquad c = \frac{a}{r}, \qquad s = \frac{b}{r},$$

则

$$\begin{bmatrix} c & s \\ -s & c \end{bmatrix} \begin{bmatrix} a \\ b \end{bmatrix} = \begin{bmatrix} r \\ 0 \end{bmatrix}.$$

对矩阵的每一列从下往上依次构造旋转，即可得到上三角矩阵 R；旋转矩阵的累积转置对应正交因子 Q。

4.2 程序验证结果

程序使用 `A3.qr_decomposition(method="givens")` 得到

$$Q = \begin{bmatrix} 0.7071 & 0.2357 & -0.6667 \\ 0 & 0.9428 & 0.3333 \\ 0.7071 & -0.2357 & 0.6667 \end{bmatrix}, \qquad R = \begin{bmatrix} 2.8284 & 2.1213 & 2.1213 \\ 0 & 2.1213 & 1.6499 \\ 0 & 0 & 1.3333 \end{bmatrix}.$$

验证误差为

表 4: 任务三验证结果

项目	数值
重构误差 $\|A - QR\|_F$	8.881784×10^{-16}
正交性误差 $\|Q^TQ - I\|_F$	3.510833×10^{-16}
SymPy 重构误差	0.000000×10^{0}

程序输出与 SymPy 的 QR 分解结果在四位小数显示下完全一致，且两个误差均处于 10^{-16} 量级，说明 Givens 旋转实现正确。

4.3 反思总结

Givens 旋转的优点是可以针对单个元素进行局部消元，特别适合稀疏矩阵或只需改变局部结构的情形。相比 Householder 反射，Givens 方法在稠密小矩阵上可能步骤更多，但过程直观，且便于观察每次旋转如何把某个下三角元素消为零。本次任务中，Givens 分解结果的残差和正交性误差都非常小，说明旋转矩阵的累积方向与 $A = QR$ 的约定保持一致。

5 完整运行输出摘要

下面给出本地运行 `python main.py` 后的输出摘要，作为报告中数值结果的来源。为了避免任务一的 SymPy 阶梯形巨大整数输出影响正文阅读，正文表格只摘录关键验证指标；完整程序仍保留在附录中。

Listing 1: main.py 运行输出

```
============================================================
Task 1
Original matrix A (10x8):
[    3.0000     4.0000    -8.0000    -1.0000     7.0000     6.0000     3.0000     0.0000]
[    6.0000     2.0000     9.0000    -3.0000     7.0000    -5.0000     0.0000    -5.0000]
[   -6.0000    -1.0000     8.0000    -5.0000     0.0000    -6.0000    -7.0000     1.0000]
[    6.0000     8.0000    -6.0000     2.0000     4.0000     1.0000    -3.0000     8.0000]
[    6.0000     5.0000     7.0000    -1.0000    -8.0000     8.0000    -9.0000    -7.0000]
[    3.0000    -9.0000     6.0000     1.0000    -2.0000     1.0000    -7.0000    -3.0000]
[    9.0000    -2.0000    -2.0000    -5.0000     8.0000     5.0000    -7.0000    -7.0000]
[    1.0000     7.0000     6.0000    -6.0000     0.0000     8.0000     0.0000    -6.0000]
[    8.0000     1.0000     8.0000    -3.0000     8.0000     9.0000     0.0000     5.0000]
[   -7.0000     3.0000     1.0000     9.0000    -2.0000     0.0000    -4.0000    -3.0000]

Row echelon form (pivoting=True):
[    9.0000    -2.0000    -2.0000    -5.0000     8.0000     5.0000    -7.0000    -7.0000]
[    0.0000     9.3333    -4.6667     5.3333    -1.3333    -2.3333     1.6667    12.6667]
[    0.0000     0.0000    12.0000    -1.5714     2.1429    -7.5000     4.0714    -4.8571]
[    0.0000     0.0000     0.0000    -8.2837    -1.6131    15.3958    -3.8482   -11.0437]
[    0.0000     0.0000     0.0000     0.0000   -14.5250    13.8468    -9.4684    -6.5674]
[    0.0000     0.0000     0.0000     0.0000     0.0000    15.7538   -14.0806   -17.6896]
[    0.0000     0.0000     0.0000     0.0000     0.0000     0.0000   -19.4138     0.4436]
[    0.0000     0.0000     0.0000     0.0000     0.0000     0.0000     0.0000    26.4275]
```

```
[    0.0000     0.0000     0.0000     0.0000     0.0000     0.0000     0.0000     0.0000]
[    0.0000     0.0000     0.0000     0.0000     0.0000     0.0000     0.0000     0.0000]
Number of row swaps: 7

Sympy row echelon form (for comparison):
[    3.0000     4.0000    -8.0000    -1.0000     7.0000     6.0000     3.0000     0.0000]
[    0.0000   -18.0000    75.0000    -3.0000   -21.0000   -51.0000   -18.0000   -15.0000]
[    0.0000     0.0000 -1143.0000   441.0000  -315.0000   747.0000   432.0000   261.0000]
[    0.0000     0.0000     0.0000 -26946.0000 43740.0000 15309.0000 17901.0000 -35262.0000]
[    0.0000     0.0000     0.0000     0.0000 20594532852.0000 -24142498578.0000 7224251490.0000 15578385912.0000]
[    0.0000     0.0000     0.0000     0.0000     0.0000 75704789307487076352.0000 661159964141656211456.0000
     -491984122824002437120.0000]
[    0.0000     0.0000     0.0000     0.0000     0.0000     0.0000 -289030607397892380998011835018158509391872.0000
     2025516889268083938499398670398476045516800.0000]
[    0.0000     0.0000     0.0000     0.0000     0.0000     0.0000     0.0000
     427319893111513121055983005184966892708995081129611870833606549372027712524045516 8.0000]
[    0.0000     0.0000     0.0000     0.0000     0.0000     0.0000     0.0000     0.0000]
[    0.0000     0.0000     0.0000     0.0000     0.0000     0.0000     0.0000     0.0000]
Our rank: 8, Sympy rank: 8

============================================================
Task 2
Matrix A:
[    3.0000    14.0000     9.0000]
[    6.0000    43.0000     3.0000]
[    6.0000    22.0000    15.0000]

Householder Q:
[   -0.3333     0.1333     0.9333]
[   -0.6667    -0.7333    -0.1333]
[   -0.6667     0.6667    -0.3333]

Householder R:
[   -9.0000   -48.0000   -15.0000]
[    0.0000   -15.0000     9.0000]
[    0.0000     0.0000     3.0000]

Householder: residual=9.503924e-15, orthogonality=4.191000e-16

Sympy Q:
[      0.3333      -0.1333       0.9333]
[      0.6667       0.7333      -0.1333]
[      0.6667      -0.6667      -0.3333]
Sympy R:
[      9.0000      48.0000      15.0000]
[      0.0000      15.0000      -9.0000]
[      0.0000       0.0000       3.0000]
Sympy QR:    residual=0.000000e+00

============================================================
Task 3
Matrix A:
[    2.0000     2.0000     1.0000]
[    0.0000     2.0000     2.0000]
[    2.0000     1.0000     2.0000]

Givens Q:
[    0.7071     0.2357    -0.6667]
[    0.0000     0.9428     0.3333]
[    0.7071    -0.2357     0.6667]
```

```
Givens R:
[    2.8284     2.1213     2.1213]
[    0.0000     2.1213     1.6499]
[    0.0000     0.0000     1.3333]

Givens:      residual=8.881784e-16, orthogonality=3.510833e-16

Sympy Q:
[      0.7071       0.2357      -0.6667]
[      0.0000       0.9428       0.3333]
[      0.7071      -0.2357       0.6667]
Sympy R:
[      2.8284       2.1213       2.1213]
[      0.0000       2.1213       1.6499]
[      0.0000       0.0000       1.3333]
Sympy QR:    residual=0.000000e+00
```

A main.py 完整源码

Listing 2: main.py 完整源码

```python
import math
import random

try:
    import sympy as sp
except ImportError:
    sp = None

from matlibrary import Matrix


def _frob_norm(data):
    total = 0.0
    for row in data:
        for val in row:
            total += val * val
    return math.sqrt(total)


def _diff_frob(a, b):
    if not a:
        return 0.0
    total = 0.0
    rows = len(a)
    cols = len(a[0])
    for i in range(rows):
        for j in range(cols):
            diff = a[i][j] - b[i][j]
            total += diff * diff
    return math.sqrt(total)


def _residual_error(A, Q, R):
    A_data = A.to_float_data()
    QR_data = (Q * R).to_float_data()
    return _diff_frob(A_data, QR_data)

```

```python

def _orthogonality_error(Q):
    m, _ = Q.dim
    QtQ = Q.transpose() * Q
    I = Matrix.identity(m, tolerance=Q.tolerance, dtype="float")
    return _diff_frob(QtQ.to_float_data(), I.to_float_data())


def _sympy_qr(data):
    if sp is None:
        print("sympy is not installed. Install with: pip install sympy")
        return None
    A_sym = sp.Matrix(data)
    Q_sym, R_sym = A_sym.QRdecomposition()
    return A_sym, Q_sym, R_sym


def _sympy_echelon(data):
    """Return sympy row echelon form (not reduced) of a matrix."""
    if sp is None:
        return None
    A_sym = sp.Matrix(data)
    return A_sym.echelon_form()


def _format_matrix(data, precision=6, width=12):
    if not data:
        return "[]"
    lines = []
    for row in data:
        parts = [f"{val:>{width}.{precision}f}" for val in row]
        lines.append("[" + " ".join(parts) + "]")
    return "\n".join(lines)


def _sympy_to_float_list(sympy_mat):
    rows, cols = sympy_mat.shape
    return [[float(sympy_mat[i, j]) for j in range(cols)] for i in range(rows)]


def main():
    print("=" * 60)
    print("Task 1")

    rng = random.Random(0)
    rows, cols = 10, 8
    data = [[rng.randint(-9, 9) for _ in range(cols)] for _ in range(rows)]
    A = Matrix(data, tolerance=1e-10, dtype="float")

    print("Original matrix A (10x8):")
    print(A)
    print()

    ref, swaps = A.gauss_elimination(pivoting=True)
    print("Row echelon form (pivoting=True):")
    print(ref)
    print(f"Number of row swaps: {swaps}")
    print()

    if sp is not None:
        sympy_ech = _sympy_echelon(data)
```

```
        if sympy_ech is not None:
            print("Sympy row echelon form (for comparison):")
            print(_format_matrix(_sympy_to_float_list(sympy_ech), precision=4, width=10))
            our_rank = ref.rank()
            sympy_rank = sympy_ech.rank()
            print(f"Our rank: {our_rank}, Sympy rank: {sympy_rank}")
    else:
        print("sympy not available for verification.")
    print()

    print("=" * 60)
    print("Task 2")

    A2_data = [[3, 14, 9],
               [6, 43, 3],
               [6, 22, 15]]
    A2 = Matrix(A2_data, tolerance=1e-10, dtype="float")
    Q2_h, R2_h = A2.qr_decomposition(method="householder")

    print("Matrix A:")
    print(A2)
    print()
    print("Householder Q:")
    print(Q2_h)
    print()
    print("Householder R:")
    print(R2_h)
    print()

    err_h = _residual_error(A2, Q2_h, R2_h)
    orth_h = _orthogonality_error(Q2_h)
    print(f"Householder: residual={err_h:.6e}, orthogonality={orth_h:.6e}")
    print()

    sympy_result2 = _sympy_qr(A2_data)
    if sympy_result2 is not None:
        A2_sym, Q2_sym, R2_sym = sympy_result2
        diff = A2_sym - Q2_sym * R2_sym
        err_sym = math.sqrt(sum(float(x) * float(x) for x in diff))
        print("Sympy Q:")
        print(_format_matrix(_sympy_to_float_list(Q2_sym), precision=4, width=12))
        print("Sympy R:")
        print(_format_matrix(_sympy_to_float_list(R2_sym), precision=4, width=12))
        print(f"Sympy QR:    residual={err_sym:.6e}")
    print()

    print("=" * 60)
    print("Task 3")

    A3_data = [[2, 2, 1],
               [0, 2, 2],
               [2, 1, 2]]
    A3 = Matrix(A3_data, tolerance=1e-10, dtype="float")
    Q3_g, R3_g = A3.qr_decomposition(method="givens")

    print("Matrix A:")
    print(A3)
    print()
    print("Givens Q:")
    print(Q3_g)
    print()
```

```
    print("Givens R:")
    print(R3_g)
    print()

    err_g = _residual_error(A3, Q3_g, R3_g)
    orth_g = _orthogonality_error(Q3_g)
    print(f"Givens:      residual={err_g:.6e}, orthogonality={orth_g:.6e}")
    print()

    # sympy 验证
    sympy_result3 = _sympy_qr(A3_data)
    if sympy_result3 is not None:
        A3_sym, Q3_sym, R3_sym = sympy_result3
        diff = A3_sym - Q3_sym * R3_sym
        err_sym = math.sqrt(sum(float(x) * float(x) for x in diff))
        print("Sympy Q:")
        print(_format_matrix(_sympy_to_float_list(Q3_sym), precision=4, width=12))
        print("Sympy R:")
        print(_format_matrix(_sympy_to_float_list(R3_sym), precision=4, width=12))
        print(f"Sympy QR:    residual={err_sym:.6e}")
    print()


if __name__ == "__main__":
    main()
```

B matlibrary.py 完整源码

Listing 3: matlibrary.py 完整源码

```
import copy
import math
from fractions import Fraction


def _is_number(value):
    return isinstance(value, (int, float, Fraction))

class Matrix:
    """
    Unified matrix class that consolidates features from:
    - matrix.py (data/dim architecture, indexing, Jordan form, helpers)
    - hw1/cholesky.py (Cholesky, Gram-Schmidt, QR, QR eigenvalues)
    - hw1/incomplete_lu_matrix_class_maybe.py (LU, exact arithmetic utilities)
    - hw2/singular_value_decomposition.py (Jacobi eigenpairs + SVD)

    The class supports two storage modes:
    - dtype="fraction" for exact rational arithmetic where possible
    - dtype="float" for numerical linear algebra
    """

    def __init__(self, data=None, dim=None, init_value=0, tolerance=1e-10, dtype="fraction"):
        if dtype not in {"fraction", "float"}:
            raise ValueError("dtype must be 'fraction' or 'float'.")

        self.dtype = dtype
        self.tolerance = float(tolerance)

```

```
        if data is None and dim is None:
            raise ValueError("Either data or dim must be provided.")

        if isinstance(data, Matrix):
            data = copy.deepcopy(data.data)

        if data is not None:
            if not isinstance(data, list):
                raise TypeError("data must be a nested list or Matrix.")

            if len(data) == 0:
                self.data = []
                self.dim = (0, 0)
                self.init_value = init_value
                return

            row_len = None
            parsed = []
            for row in data:
                if not isinstance(row, list):
                    raise TypeError("Every row in data must be a list.")
                if row_len is None:
                    row_len = len(row)
                elif len(row) != row_len:
                    raise ValueError("All rows in data must have the same length.")

                parsed_row = [self._cast_value(x) for x in row]
                parsed.append(parsed_row)

            self.data = parsed
            self.dim = (len(parsed), row_len)
            self.init_value = init_value
            return

        if not isinstance(dim, tuple) or len(dim) != 2:
            raise TypeError("dim must be a tuple (rows, cols).")
        m, n = dim
        if not (isinstance(m, int) and isinstance(n, int)):
            raise TypeError("dim entries must be integers.")
        if m < 0 or n < 0:
            raise ValueError("dim entries must be non-negative.")

        fill = self._cast_value(init_value)
        self.data = [[fill for _ in range(n)] for _ in range(m)]
        self.dim = (m, n)
        self.init_value = init_value

    def _cast_value(self, value):
        if not _is_number(value):
            raise TypeError("Matrix elements must be int, float, or Fraction.")
        if self.dtype == "fraction":
            return value if isinstance(value, Fraction) else Fraction(value)
        return float(value)

    @staticmethod
    def _near_zero(value, tol):
        if isinstance(value, Fraction):
            return value == 0
        return abs(float(value)) <= tol

    @staticmethod
```

```
    def _dot(v1, v2):
        return sum(v1[i] * v2[i] for i in range(len(v1)))

    @classmethod
    def _norm(cls, vec):
        return math.sqrt(max(0.0, cls._dot(vec, vec)))

    @classmethod
    def _normalize_vec(cls, vec, tol):
        nrm = cls._norm(vec)
        if nrm <= tol:
            raise ValueError("Cannot normalize a near-zero vector.")
        return [x / nrm for x in vec]

    @staticmethod
    def _mat_vec_mul(mat, vec):
        return [sum(mat[i][j] * vec[j] for j in range(len(vec))) for i in range(len(mat))]

    @classmethod
    def _complete_orthonormal_basis(cls, base_vectors, dimension, tol):
        basis = []
        for vec in base_vectors:
            v = vec[:]
            for q in basis:
                coeff = cls._dot(v, q)
                v = [v[i] - coeff * q[i] for i in range(dimension)]
            nrm = cls._norm(v)
            if nrm > tol:
                basis.append([x / nrm for x in v])

        for idx in range(dimension):
            v = [0.0] * dimension
            v[idx] = 1.0
            for q in basis:
                coeff = cls._dot(v, q)
                v = [v[i] - coeff * q[i] for i in range(dimension)]
            nrm = cls._norm(v)
            if nrm > tol:
                basis.append([x / nrm for x in v])
            if len(basis) == dimension:
                break

        if len(basis) < dimension:
            raise ValueError("Failed to complete an orthonormal basis.")
        return basis

    def _is_symmetric(self):
        m, n = self.dim
        if m != n:
            return False
        for i in range(m):
            for j in range(i + 1, n):
                if abs(float(self.data[i][j] - self.data[j][i])) > self.tolerance:
                    return False
        return True

    def to_float_data(self):
        return [[float(x) for x in row] for row in self.data]

    @staticmethod
    def method_comparison():
```

```
        """
        Returns markdown-style notes comparing methods for the same task.
        """
        return (
            "# Method Comparison\n"
            "- Gaussian elimination: partial pivoting is more stable than no-pivot elimination; use pivoting by default.\n"
            "- QR decomposition: Modified Gram-Schmidt is usually better than Classical Gram-Schmidt in finite precision.\n"
            "- QR with Householder is typically the most numerically stable general-purpose QR.\n"
            "- Eigenvalues: Jacobi is preferred for real symmetric matrices (stable, gives orthonormal eigenvectors).\n"
            "- Eigenvalues: QR iteration is more general for non-symmetric matrices, but eigenvectors are not returned here.\n"
            "- SVD: using A^T A for V and then U = A v_i / sigma_i is more robust than independently matching U from A A^T.\n"
            "- Cholesky is best for symmetric positive-definite matrices; LU is more general."
        )

    def shape(self):
        return self.dim

    def copy(self):
        return Matrix(
            data=copy.deepcopy(self.data),
            tolerance=self.tolerance,
            dtype=self.dtype,
        )

    def reshape(self, newdim):
        if not isinstance(newdim, tuple) or len(newdim) != 2:
            raise TypeError("newdim must be a tuple (rows, cols).")
        nr, nc = newdim
        if not (isinstance(nr, int) and isinstance(nc, int)):
            raise TypeError("newdim entries must be integers.")
        if nr < 0 or nc < 0:
            raise ValueError("newdim entries must be non-negative.")

        r, c = self.dim
        if r * c != nr * nc:
            raise ValueError("Cannot reshape to a different total number of elements.")

        flat = [self.data[i][j] for i in range(r) for j in range(c)]
        idx = 0
        out = []
        for _ in range(nr):
            row = []
            for _ in range(nc):
                row.append(flat[idx])
                idx += 1
            out.append(row)
        return Matrix(out, tolerance=self.tolerance, dtype=self.dtype)

    def transpose(self):
        r, c = self.dim
        if r == 0:
            return Matrix(dim=(c, r), tolerance=self.tolerance, dtype=self.dtype)
        out = [[self.data[i][j] for i in range(r)] for j in range(c)]
        return Matrix(out, tolerance=self.tolerance, dtype=self.dtype)

    def T(self):
```

```
        return self.transpose()

    @classmethod
    def identity(cls, n, tolerance=1e-10, dtype="fraction"):
        if not isinstance(n, int) or n < 0:
            raise ValueError("n must be a non-negative integer.")
        one = Fraction(1) if dtype == "fraction" else 1.0
        zero = Fraction(0) if dtype == "fraction" else 0.0
        out = [[one if i == j else zero for j in range(n)] for i in range(n)]
        return cls(out, tolerance=tolerance, dtype=dtype)

    def sum(self, axis=None):
        r, c = self.dim
        if axis not in {None, 0, 1}:
            raise ValueError("axis must be None, 0, or 1.")

        if axis is None:
            total = 0
            for row in self.data:
                for val in row:
                    total += val
            return Matrix([[total]], tolerance=self.tolerance, dtype=self.dtype)

        if axis == 0:
            out = [[sum(self.data[i][j] for i in range(r)) for j in range(c)]]
            return Matrix(out, tolerance=self.tolerance, dtype=self.dtype)

        out = [[sum(self.data[i][j] for j in range(c))] for i in range(r)]
        return Matrix(out, tolerance=self.tolerance, dtype=self.dtype)

    def concatenate(self, other, index=0):
        if not isinstance(other, Matrix):
            raise TypeError("other must be a Matrix.")

        out_dtype = "fraction" if self.dtype == other.dtype == "fraction" else "float"
        out_tol = max(self.tolerance, other.tolerance)

        if index == 0:
            if self.dim[1] != other.dim[1]:
                raise ValueError("For row concatenation, column counts must match.")
            data = copy.deepcopy(self.data) + copy.deepcopy(other.data)
            return Matrix(data, tolerance=out_tol, dtype=out_dtype)

        if index == 1:
            if self.dim[0] != other.dim[0]:
                raise ValueError("For column concatenation, row counts must match.")
            data = [
                copy.deepcopy(self.data[i]) + copy.deepcopy(other.data[i])
                for i in range(self.dim[0])
            ]
            return Matrix(data, tolerance=out_tol, dtype=out_dtype)

        raise ValueError("index must be 0 (rows) or 1 (cols).")

    def kronecker_product(self, other):
        if not isinstance(other, Matrix):
            raise TypeError("other must be a Matrix.")
        m1, n1 = self.dim
        m2, n2 = other.dim
        out_dtype = "fraction" if self.dtype == other.dtype == "fraction" else "float"
        out_tol = max(self.tolerance, other.tolerance)
```

```
268
269         data = [[0 for _ in range(n1 * n2)] for _ in range(m1 * m2)]
270         for i in range(m1):
271             for j in range(n1):
272                 for p in range(m2):
273                     for q in range(n2):
274                         data[i * m2 + p][j * n2 + q] = self.data[i][j] * other.data[p][q]
275         return Matrix(data, tolerance=out_tol, dtype=out_dtype)
276
277     def Kronecker_product(self, other):
278         return self.kronecker_product(other)
279
280     def __getitem__(self, key):
281         if not isinstance(key, tuple) or len(key) != 2:
282             raise TypeError("Index must be a tuple (row_key, col_key).")
283         row_key, col_key = key
284         r, c = self.dim
285
286         def _resolve(selector, size):
287             if isinstance(selector, int):
288                 idx = selector + size if selector < 0 else selector
289                 if idx < 0 or idx >= size:
290                     raise IndexError("Index out of range.")
291                 return [idx], True
292             if isinstance(selector, slice):
293                 return list(range(*selector.indices(size))), False
294             raise TypeError("Each index must be int or slice.")
295
296         row_idx, row_single = _resolve(row_key, r)
297         col_idx, col_single = _resolve(col_key, c)
298
299         if row_single and col_single:
300             return self.data[row_idx[0]][col_idx[0]]
301
302         if len(row_idx) == 0:
303             return Matrix(dim=(0, len(col_idx)), tolerance=self.tolerance, dtype=self.dtype)
304
305         data = [[self.data[i][j] for j in col_idx] for i in row_idx]
306         return Matrix(data, tolerance=self.tolerance, dtype=self.dtype)
307
308     def __setitem__(self, key, value):
309         if not isinstance(key, tuple) or len(key) != 2:
310             raise TypeError("Index must be a tuple (row_key, col_key).")
311         row_key, col_key = key
312         r, c = self.dim
313
314         def _resolve(selector, size):
315             if isinstance(selector, int):
316                 idx = selector + size if selector < 0 else selector
317                 if idx < 0 or idx >= size:
318                     raise IndexError("Index out of range.")
319                 return [idx], True
320             if isinstance(selector, slice):
321                 return list(range(*selector.indices(size))), False
322             raise TypeError("Each index must be int or slice.")
323
324         row_idx, row_single = _resolve(row_key, r)
325         col_idx, col_single = _resolve(col_key, c)
326
327         if row_single and col_single:
328             if not _is_number(value):
```

```
                raise TypeError("Scalar assignment requires a numeric value.")
            self.data[row_idx[0]][col_idx[0]] = self._cast_value(value)
            return

        if not isinstance(value, Matrix):
            raise TypeError("Slice assignment requires a Matrix value.")
        if value.dim != (len(row_idx), len(col_idx)):
            raise ValueError("Assigned matrix shape does not match target slice shape.")

        for i, rr in enumerate(row_idx):
            for j, cc in enumerate(col_idx):
                self.data[rr][cc] = self._cast_value(value.data[i][j])

    def __len__(self):
        return self.dim[0] * self.dim[1]

    def __str__(self):
        r, c = self.dim
        if r == 0:
            return "[]"
        if c == 0:
            return "\n".join(["[]" for _ in range(r)])
        lines = []
        for row in self.data:
            lines.append("[" + " ".join(f"{float(x):>10.4f}" for x in row) + "]")
        return "\n".join(lines)

    def num_product(self, n):
        if not _is_number(n):
            raise TypeError("Scalar multiplier must be numeric.")
        out = [[val * n for val in row] for row in self.data]
        out_dtype = self.dtype if self.dtype == "fraction" and not isinstance(n, float) else "float"
        return Matrix(out, tolerance=self.tolerance, dtype=out_dtype)

    def __rmul__(self, other):
        if _is_number(other):
            return self.num_product(other)
        return NotImplemented

    def __mul__(self, other):
        if _is_number(other):
            return self.num_product(other)

        if not isinstance(other, Matrix):
            raise TypeError("Matrix multiplication requires a Matrix or a scalar.")
        if self.dim[1] != other.dim[0]:
            raise ValueError("Incompatible shapes for matrix multiplication.")

        m, k = self.dim
        _, n = other.dim
        out = []
        for i in range(m):
            row = []
            for j in range(n):
                val = 0
                for t in range(k):
                    val += self.data[i][t] * other.data[t][j]
                row.append(val)
            out.append(row)

        out_dtype = "fraction" if self.dtype == other.dtype == "fraction" else "float"
```

```
        out_tol = max(self.tolerance, other.tolerance)
        return Matrix(out, tolerance=out_tol, dtype=out_dtype)

    def __matmul__(self, other):
        return self.__mul__(other)

    def __add__(self, other):
        if not isinstance(other, Matrix):
            raise TypeError("Addition requires a Matrix.")
        if self.dim != other.dim:
            raise ValueError("Shapes must match for addition.")
        out = [
            [self.data[i][j] + other.data[i][j] for j in range(self.dim[1])]
            for i in range(self.dim[0])
        ]
        out_dtype = "fraction" if self.dtype == other.dtype == "fraction" else "float"
        out_tol = max(self.tolerance, other.tolerance)
        return Matrix(out, tolerance=out_tol, dtype=out_dtype)

    def __sub__(self, other):
        if not isinstance(other, Matrix):
            raise TypeError("Subtraction requires a Matrix.")
        if self.dim != other.dim:
            raise ValueError("Shapes must match for subtraction.")
        out = [
            [self.data[i][j] - other.data[i][j] for j in range(self.dim[1])]
            for i in range(self.dim[0])
        ]
        out_dtype = "fraction" if self.dtype == other.dtype == "fraction" else "float"
        out_tol = max(self.tolerance, other.tolerance)
        return Matrix(out, tolerance=out_tol, dtype=out_dtype)

    def __pow__(self, n):
        if not isinstance(n, int):
            raise TypeError("Exponent must be an integer.")
        if n < 0:
            raise ValueError("Negative powers are not supported directly; use inverse() first.")
        if self.dim[0] != self.dim[1]:
            raise ValueError("Only square matrices can be exponentiated.")

        size = self.dim[0]
        result = Matrix.identity(size, tolerance=self.tolerance, dtype=self.dtype)
        base = self.copy()
        exp = n
        while exp > 0:
            if exp % 2 == 1:
                result = result * base
            base = base * base
            exp //= 2
        return result

    def gauss_elimination(self, pivoting=True):
        """
        Row-echelon reduction by Gaussian elimination.
        Returns: (row_echelon_matrix, swap_count)

        Better method note:
        - pivoting=True is generally better numerically than pivoting=False.
        """
        mat = copy.deepcopy(self.data)
        rows, cols = self.dim
```

```
        swap_count = 0
        tol = self.tolerance
        zero = Fraction(0) if self.dtype == "fraction" else 0.0

        r = 0
        for c in range(cols):
            if r >= rows:
                break

            pivot_row = None
            if pivoting:
                best = 0.0
                for i in range(r, rows):
                    if not self._near_zero(mat[i][c], tol):
                        score = abs(float(mat[i][c]))
                        if score > best:
                            best = score
                            pivot_row = i
            else:
                for i in range(r, rows):
                    if not self._near_zero(mat[i][c], tol):
                        pivot_row = i
                        break

            if pivot_row is None:
                continue

            if pivot_row != r:
                mat[r], mat[pivot_row] = mat[pivot_row], mat[r]
                swap_count += 1

            pivot = mat[r][c]
            for i in range(r + 1, rows):
                if self._near_zero(mat[i][c], tol):
                    continue
                factor = mat[i][c] / pivot
                for j in range(c, cols):
                    mat[i][j] -= factor * mat[r][j]
                    if self._near_zero(mat[i][j], tol):
                        mat[i][j] = zero
            r += 1

        return Matrix(mat, tolerance=self.tolerance, dtype=self.dtype), swap_count

    def determinant(self):
        if self.dim[0] != self.dim[1]:
            raise ValueError("Determinant is defined only for square matrices.")
        ref, swaps = self.gauss_elimination(pivoting=True)
        n = self.dim[0]
        det = Fraction(1) if self.dtype == "fraction" else 1.0
        for i in range(n):
            det *= ref.data[i][i]
        if swaps % 2 == 1:
            det = -det
        if self._near_zero(det, self.tolerance):
            return Fraction(0) if self.dtype == "fraction" else 0.0
        return det

    def det(self):
        return self.determinant()

```

```
512     def inverse(self):
513         """
514         Gauss-Jordan inverse with partial pivoting.
515         Better than a no-pivot variant for numerical stability.
516         """
517         n, m = self.dim
518         if n != m:
519             raise ValueError("Inverse is defined only for square matrices.")
520 
521         tol = self.tolerance
522         zero = Fraction(0) if self.dtype == "fraction" else 0.0
523         one = Fraction(1) if self.dtype == "fraction" else 1.0
524 
525         aug = [
526             copy.deepcopy(self.data[i]) + [one if i == j else zero for j in range(n)]
527             for i in range(n)
528         ]
529 
530         for col in range(n):
531             pivot_row = col
532             best = abs(float(aug[col][col]))
533             for r in range(col + 1, n):
534                 score = abs(float(aug[r][col]))
535                 if score > best:
536                     best = score
537                     pivot_row = r
538 
539             if self._near_zero(aug[pivot_row][col], tol):
540                 raise ValueError("Matrix is singular and cannot be inverted.")
541 
542             if pivot_row != col:
543                 aug[col], aug[pivot_row] = aug[pivot_row], aug[col]
544 
545             pivot = aug[col][col]
546             for j in range(col, 2 * n):
547                 aug[col][j] /= pivot
548 
549             for r in range(n):
550                 if r == col:
551                     continue
552                 factor = aug[r][col]
553                 if self._near_zero(factor, tol):
554                     continue
555                 for j in range(col, 2 * n):
556                     aug[r][j] -= factor * aug[col][j]
557                     if self._near_zero(aug[r][j], tol):
558                         aug[r][j] = zero
559 
560         inv = [[aug[i][j + n] for j in range(n)] for i in range(n)]
561         return Matrix(inv, tolerance=self.tolerance, dtype=self.dtype)
562 
563     def rank(self):
564         ref, _ = self.gauss_elimination(pivoting=True)
565         rank_val = 0
566         for i in range(ref.dim[0]):
567             if any(not self._near_zero(ref.data[i][j], self.tolerance) for j in range(ref.dim[1])):
568                 rank_val += 1
569         return rank_val
570 
571     def lu_decomposition(self, pivoting=True):
572         """
```

```
        Doolittle LU decomposition.

        Returns:
        - pivoting=True: (P, L, U)
        - pivoting=False: (L, U)

        Better method note:
        - pivoting=True is usually better and more robust.
        """
        n, m = self.dim
        if n != m:
            raise ValueError("LU decomposition requires a square matrix.")

        tol = self.tolerance
        zero = Fraction(0) if self.dtype == "fraction" else 0.0
        one = Fraction(1) if self.dtype == "fraction" else 1.0

        U = copy.deepcopy(self.data)
        L = [[zero for _ in range(n)] for _ in range(n)]
        P = [[one if i == j else zero for j in range(n)] for i in range(n)]

        for i in range(n):
            L[i][i] = one

        for k in range(n):
            pivot_row = k
            if pivoting:
                best = abs(float(U[k][k]))
                for r in range(k + 1, n):
                    score = abs(float(U[r][k]))
                    if score > best:
                        best = score
                        pivot_row = r

            if self._near_zero(U[pivot_row][k], tol):
                raise ValueError("Matrix is singular or LU requires pivoting at a zero pivot.")

            if pivot_row != k:
                U[k], U[pivot_row] = U[pivot_row], U[k]
                P[k], P[pivot_row] = P[pivot_row], P[k]
                for c in range(k):
                    L[k][c], L[pivot_row][c] = L[pivot_row][c], L[k][c]

            for r in range(k + 1, n):
                factor = U[r][k] / U[k][k]
                L[r][k] = factor
                for c in range(k, n):
                    U[r][c] -= factor * U[k][c]
                    if self._near_zero(U[r][c], tol):
                        U[r][c] = zero

        L_mat = Matrix(L, tolerance=self.tolerance, dtype=self.dtype)
        U_mat = Matrix(U, tolerance=self.tolerance, dtype=self.dtype)
        if pivoting:
            P_mat = Matrix(P, tolerance=self.tolerance, dtype=self.dtype)
            return P_mat, L_mat, U_mat
        return L_mat, U_mat

    def cholesky_decomposition(self):
        """
        Cholesky decomposition A = L * L^T for SPD matrices.
```

```
        """
        n, m = self.dim
        if n != m:
            raise ValueError("Cholesky decomposition requires a square matrix.")
        if not self._is_symmetric():
            raise ValueError("Cholesky decomposition requires a symmetric matrix.")

        A = self.to_float_data()
        tol = self.tolerance
        L = [[0.0 for _ in range(n)] for _ in range(n)]

        for i in range(n):
            for j in range(i + 1):
                s = sum(L[i][k] * L[j][k] for k in range(j))
                if i == j:
                    val = A[i][i] - s
                    if val <= tol:
                        raise ValueError("Matrix is not positive definite.")
                    L[i][j] = math.sqrt(val)
                else:
                    if abs(L[j][j]) <= tol:
                        raise ValueError("Matrix is not positive definite.")
                    L[i][j] = (A[i][j] - s) / L[j][j]

        L_mat = Matrix(L, tolerance=self.tolerance, dtype="float")
        return L_mat, L_mat.transpose()

    def gram_schmidt(self, method="mgs"):
        """
        Orthonormalize columns of A.

        method:
        - 'cgs' / 'classical': Classical Gram-Schmidt (less stable)
        - 'mgs' / 'modified': Modified Gram-Schmidt (better)
        """
        m, n = self.dim
        if m == 0 or n == 0:
            raise ValueError("Cannot run Gram-Schmidt on an empty matrix.")

        method_key = method.lower()
        if method_key not in {"cgs", "classical", "mgs", "modified"}:
            raise ValueError("method must be 'cgs'/'classical' or 'mgs'/'modified'.")

        cols = [[float(self.data[i][j]) for i in range(m)] for j in range(n)]
        q_cols = []
        tol = self.tolerance

        if method_key in {"cgs", "classical"}:
            for col in cols:
                v = col[:]
                for q in q_cols:
                    proj = self._dot(col, q)
                    v = [v[i] - proj * q[i] for i in range(m)]
                norm_v = self._norm(v)
                if norm_v <= tol:
                    raise ValueError("Columns are linearly dependent; CGS failed.")
                q_cols.append([x / norm_v for x in v])
        else:
            for col in cols:
                v = col[:]
                for q in q_cols:
```

```
                    proj = self._dot(v, q)
                    v = [v[i] - proj * q[i] for i in range(m)]
                norm_v = self._norm(v)
                if norm_v <= tol:
                    raise ValueError("Columns are linearly dependent; MGS failed.")
                q_cols.append([x / norm_v for x in v])

        q_data = [[q_cols[j][i] for j in range(n)] for i in range(m)]
        return Matrix(q_data, tolerance=self.tolerance, dtype="float")

    def _qr_householder(self):
        m, n = self.dim
        if m == 0 or n == 0:
            raise ValueError("Cannot run QR on an empty matrix.")

        R = self.to_float_data()
        Q = Matrix.identity(m, tolerance=self.tolerance, dtype="float").to_float_data()
        k_max = min(m, n)
        tol = self.tolerance

        for k in range(k_max):
            x = [R[i][k] for i in range(k, m)]
            norm_x = self._norm(x)
            if norm_x <= tol:
                continue

            sign = -1.0 if x[0] < 0 else 1.0
            v = x[:]
            v[0] += sign * norm_x
            norm_v = self._norm(v)
            if norm_v <= tol:
                continue
            v = [val / norm_v for val in v]

            for j in range(k, n):
                dot_vr = sum(v[i] * R[k + i][j] for i in range(len(v)))
                for i in range(len(v)):
                    R[k + i][j] -= 2.0 * v[i] * dot_vr

            for i in range(m):
                dot_qv = sum(v[t] * Q[i][k + t] for t in range(len(v)))
                for t in range(len(v)):
                    Q[i][k + t] -= 2.0 * v[t] * dot_qv

        for i in range(m):
            for j in range(min(i, n)):
                if abs(R[i][j]) <= tol:
                    R[i][j] = 0.0

        return (
            Matrix(Q, tolerance=self.tolerance, dtype="float"),
            Matrix(R, tolerance=self.tolerance, dtype="float"),
        )

    def _qr_givens(self):
        m, n = self.dim
        if m == 0 or n == 0:
            raise ValueError("Cannot run QR on an empty matrix.")

        R = self.to_float_data()
        Q = Matrix.identity(m, tolerance=self.tolerance, dtype="float").to_float_data()
```

```
        tol = self.tolerance

        for j in range(n):
            for i in range(m - 1, j, -1):
                b = R[i][j]
                if abs(b) <= tol:
                    continue
                a = R[i - 1][j]
                r = math.hypot(a, b)
                if r <= tol:
                    continue
                c = a / r
                s = b / r

                for k in range(j, n):
                    temp = c * R[i - 1][k] + s * R[i][k]
                    R[i][k] = -s * R[i - 1][k] + c * R[i][k]
                    R[i - 1][k] = temp

                # Update Q so that A = Q * R remains true.
                for k in range(m):
                    q_im1 = Q[k][i - 1]
                    q_i = Q[k][i]
                    Q[k][i - 1] = c * q_im1 + s * q_i
                    Q[k][i] = -s * q_im1 + c * q_i

        for i in range(m):
            for j in range(min(i, n)):
                if abs(R[i][j]) <= tol:
                    R[i][j] = 0.0

        return (
            Matrix(Q, tolerance=self.tolerance, dtype="float"),
            Matrix(R, tolerance=self.tolerance, dtype="float"),
        )

    def qr_decomposition(self, method="householder"):
        """
        QR decomposition A = Q * R.

        method:
        - 'householder' (recommended)
        - 'givens'
        - 'mgs' / 'modified'
        - 'cgs' / 'classical'
        """
        method_key = method.lower()

        if method_key in {"householder", "hh"}:
            return self._qr_householder()

        if method_key in {"givens", "g"}:
            return self._qr_givens()

        if method_key in {"mgs", "modified", "cgs", "classical"}:
            Q = self.gram_schmidt(method=method_key)
            R = Q.transpose() * Matrix(self.to_float_data(), tolerance=self.tolerance, dtype="float")

            r_data = R.to_float_data()
            rows_r, cols_r = R.dim
            for i in range(rows_r):
```

```
                for j in range(cols_r):
                    if i > j and abs(r_data[i][j]) <= self.tolerance:
                        r_data[i][j] = 0.0
            return Q, Matrix(r_data, tolerance=self.tolerance, dtype="float")

        raise ValueError("Unknown QR method.")

    def eigenvalues_qr(self, iterations=100, qr_method="householder"):
        """
        Approximate eigenvalues via unshifted QR iteration.
        Better for general matrices than Jacobi (which is symmetric-only).
        """
        n, m = self.dim
        if n != m:
            raise ValueError("Eigenvalues are defined only for square matrices.")

        current = Matrix(self.to_float_data(), tolerance=self.tolerance, dtype="float")
        for _ in range(iterations):
            Q, R = current.qr_decomposition(method=qr_method)
            current = R * Q

        vals = [float(current.data[i][i]) for i in range(n)]
        return [0.0 if abs(v) <= self.tolerance else v for v in vals]

    def eigenpairs_jacobi(self, max_iterations=1000, sort=True):
        """
        Jacobi eigenvalue algorithm for real symmetric matrices.
        Returns: (eigenvalues, eigenvector_matrix_with_columns_as_vectors)
        """
        n, m = self.dim
        if n != m:
            raise ValueError("Eigenvalues are defined only for square matrices.")
        if not self._is_symmetric():
            raise ValueError("Jacobi method requires a symmetric matrix.")

        A = self.to_float_data()
        V = Matrix.identity(n, tolerance=self.tolerance, dtype="float").to_float_data()
        tol = self.tolerance

        for _ in range(max_iterations):
            max_val = 0.0
            p = -1
            q = -1
            for i in range(n):
                for j in range(i + 1, n):
                    candidate = abs(A[i][j])
                    if candidate > max_val:
                        max_val = candidate
                        p = i
                        q = j

            if max_val < tol:
                break

            tau = (A[q][q] - A[p][p]) / (2.0 * A[p][q])
            sign_tau = 1.0 if tau >= 0 else -1.0
            t = sign_tau / (abs(tau) + math.sqrt(tau * tau + 1.0))
            c = 1.0 / math.sqrt(1.0 + t * t)
            s = t * c

            for i in range(n):
```

```
                if i != p and i != q:
                    aip = A[i][p]
                    aiq = A[i][q]
                    A[i][p] = c * aip - s * aiq
                    A[p][i] = A[i][p]
                    A[i][q] = c * aiq + s * aip
                    A[q][i] = A[i][q]

            app = A[p][p]
            aqq = A[q][q]
            apq = A[p][q]
            A[p][p] = c * c * app - 2.0 * s * c * apq + s * s * aqq
            A[q][q] = s * s * app + 2.0 * s * c * apq + c * c * aqq
            A[p][q] = 0.0
            A[q][p] = 0.0

            for i in range(n):
                vip = V[i][p]
                viq = V[i][q]
                V[i][p] = c * vip - s * viq
                V[i][q] = c * viq + s * vip

        eigenvalues = [A[i][i] if abs(A[i][i]) > tol else 0.0 for i in range(n)]

        if sort:
            idx = sorted(range(n), key=lambda i: eigenvalues[i], reverse=True)
            eigenvalues = [eigenvalues[i] for i in idx]
            V = [[V[r][i] for i in idx] for r in range(n)]

        return eigenvalues, Matrix(V, tolerance=self.tolerance, dtype="float")

    def eigenpairs(self, method="auto", iterations=100, max_iterations=1000, qr_method="householder"):
        """
        Unified eigen API.

        Returns:
        - (eigenvalues, eigenvectors) for Jacobi
        - (eigenvalues, None) for QR iteration
        """
        method_key = method.lower()
        if method_key == "auto":
            method_key = "jacobi" if self._is_symmetric() else "qr"

        if method_key == "jacobi":
            return self.eigenpairs_jacobi(max_iterations=max_iterations, sort=True)

        if method_key == "qr":
            return self.eigenvalues_qr(iterations=iterations, qr_method=qr_method), None

        raise ValueError("method must be 'auto', 'jacobi', or 'qr'.")

    def eigenvalues(self, method="auto", iterations=100, max_iterations=1000, qr_method="householder"):
        vals, _ = self.eigenpairs(
            method=method,
            iterations=iterations,
            max_iterations=max_iterations,
            qr_method=qr_method,
        )
        return vals

    def singular_value_decomposition(self):
```

```
        """
        Full SVD: A = U * Sigma * V^T

        Strategy:
        - Compute eigenpairs of A^T A to get V and singular values.
        - Build U via u_i = A v_i / sigma_i (better matching than independently diagonalizing A A^T).
        - Complete U to an orthonormal basis when needed.
        """
        m, n = self.dim
        tol = self.tolerance

        A_float = self.to_float_data()
        AtA = Matrix((self.transpose() * self).to_float_data(), tolerance=tol, dtype="float")
        eigvals, V = AtA.eigenpairs_jacobi(max_iterations=2000, sort=True)

        eigvals = [ev if ev > tol else 0.0 for ev in eigvals]
        singular_values = [math.sqrt(ev) for ev in eigvals]

        k = min(m, n)
        sigma_data = [[0.0 for _ in range(n)] for _ in range(m)]
        for i in range(k):
            sigma_data[i][i] = singular_values[i]

        V_data = V.to_float_data()
        v_cols = [[V_data[r][c] for r in range(n)] for c in range(n)]

        u_cols = [None for _ in range(k)]
        for i in range(k):
            sigma = singular_values[i]
            if sigma <= tol:
                continue
            Av = self._mat_vec_mul(A_float, v_cols[i])
            u_cols[i] = self._normalize_vec([x / sigma for x in Av], tol)

        known_u = [col for col in u_cols if col is not None]
        full_basis = self._complete_orthonormal_basis(known_u, m, tol)

        basis_cursor = len(known_u)
        for i in range(k):
            if u_cols[i] is None:
                u_cols[i] = full_basis[basis_cursor]
                basis_cursor += 1

        while len(u_cols) < m:
            u_cols.append(full_basis[basis_cursor])
            basis_cursor += 1

        U_data = [[u_cols[col][row] for col in range(m)] for row in range(m)]
        U = Matrix(U_data, tolerance=tol, dtype="float")
        Sigma = Matrix(sigma_data, tolerance=tol, dtype="float")
        V_T = Matrix(V_data, tolerance=tol, dtype="float").transpose()
        return U, Sigma, V_T

    def svd(self):
        return self.singular_value_decomposition()

    def jordan_normal_form(self):
        """
        Jordan normal form for matrices with rational eigenvalues.
        The implementation follows Faddeev-LeVerrier + rational root theorem.
        """
```

```
        n, m = self.dim
        if n != m:
            raise ValueError("Jordan normal form is defined only for square matrices.")
        if n == 0:
            return Matrix(data=[], tolerance=self.tolerance, dtype=self.dtype)

        A_mat = Matrix(
            data=[[Fraction(x) for x in row] for row in self.data],
            tolerance=self.tolerance,
            dtype="fraction",
        )

        c = [Fraction(0)] * (n + 1)
        c[n] = Fraction(1)

        M_mat = Matrix([[Fraction(0)] * n for _ in range(n)], tolerance=self.tolerance, dtype="fraction")
        for k in range(1, n + 1):
            c_prev = c[n - k + 1]
            c_prev_I = Matrix(
                [[c_prev if i == j else Fraction(0) for j in range(n)] for i in range(n)],
                tolerance=self.tolerance,
                dtype="fraction",
            )
            M_mat = (A_mat * M_mat) + c_prev_I
            AM = A_mat * M_mat
            trace_AM = sum(AM.data[i][i] for i in range(n))
            c[n - k] = -Fraction(1, k) * trace_AM

        def get_factors(num):
            num = abs(int(num))
            if num == 0:
                return [1]
            fac = set()
            for i in range(1, int(math.sqrt(num)) + 2):
                if num % i == 0:
                    fac.add(i)
                    fac.add(num // i)
            return list(fac)

        def evaluate_poly(poly, x):
            res = Fraction(0)
            for coeff in reversed(poly):
                res = res * x + coeff
            return res

        def synthetic_div(poly, root):
            new_poly = []
            val = Fraction(0)
            for coeff in reversed(poly):
                val = val * root + coeff
                new_poly.append(val)
            new_poly.pop()
            return new_poly[::-1]

        roots = {}
        poly = [coeff for coeff in c]

        while len(poly) > 1 and poly[0] == 0:
            roots[Fraction(0)] = roots.get(Fraction(0), 0) + 1
            poly = poly[1:]
```

```
        if len(poly) > 1:
            def lcm(a, b):
                return abs(a * b) // math.gcd(a, b)

            lcm_den = 1
            for coeff in poly:
                lcm_den = lcm(lcm_den, coeff.denominator)

            int_coeffs = [int(coeff * lcm_den) for coeff in poly]
            c0 = int_coeffs[0]
            cn = int_coeffs[-1]

            p_candidates = get_factors(c0)
            q_candidates = get_factors(cn)
            candidates = set()
            for p in p_candidates:
                for q in q_candidates:
                    candidates.add(Fraction(p, q))
                    candidates.add(Fraction(-p, q))

            for cand in list(candidates):
                while len(int_coeffs) > 1 and evaluate_poly(int_coeffs, cand) == 0:
                    roots[cand] = roots.get(cand, 0) + 1
                    int_coeffs = synthetic_div(int_coeffs, cand)

            if len(int_coeffs) > 1:
                raise ValueError("Matrix has non-rational eigenvalues; exact Jordan form is unsupported here.")

        jordan_blocks = []
        for eigenvalue, alg_mult in roots.items():
            diag_B = Matrix(
                [[-eigenvalue if i == j else Fraction(0) for j in range(n)] for i in range(n)],
                tolerance=self.tolerance,
                dtype="fraction",
            )
            B = A_mat + diag_B

            ranks = [n]
            current_B = Matrix.identity(n, tolerance=self.tolerance, dtype="fraction")

            k_max = alg_mult
            for k in range(1, k_max + 2):
                current_B = current_B * B
                ranks.append(current_B.rank())
                if len(ranks) >= 3 and ranks[-1] == ranks[-2]:
                    while len(ranks) <= k_max + 2:
                        ranks.append(ranks[-1])
                    break

            for k in range(1, alg_mult + 1):
                if k + 1 < len(ranks):
                    num_blocks = ranks[k - 1] - 2 * ranks[k] + ranks[k + 1]
                    for _ in range(num_blocks):
                        jordan_blocks.append((eigenvalue, k))

        J_data = [[Fraction(0)] * n for _ in range(n)]
        idx = 0
        for eigenvalue, block_size in jordan_blocks:
            for i in range(block_size):
                J_data[idx + i][idx + i] = eigenvalue
                if i < block_size - 1:
```

```
                    J_data[idx + i][idx + i + 1] = Fraction(1)
            idx += block_size

        return Matrix(J_data, tolerance=self.tolerance, dtype="fraction")


def I(n, tolerance=1e-10, dtype="fraction"):
    return Matrix.identity(n, tolerance=tolerance, dtype=dtype)


def concatenate(items, axis=0):
    items = list(items)
    if len(items) == 0:
        raise ValueError("No matrices to concatenate.")
    for item in items:
        if not isinstance(item, Matrix):
            raise TypeError("All items must be Matrix objects.")

    result = items[0].copy()
    for item in items[1:]:
        result = result.concatenate(item, index=axis)
    return result

def main():
    pass

if __name__ == "__main__":
    main()
```

说明

本技术报告的书写, 排版由 ChatGPT 辅助完成；报告中所分析和列出的程序代码全部为人工书写。本人在使用辅助工具时，对算法描述、数值结果、代码内容和最终表述进行了检查与修改。

同时, 为了避免不必要的麻烦, 我必须说明: 本技术报告中的代码并非单纯是本课程的产物。这一代码最早可以追溯到上学期的课程 MATH1205 的课程大作业, 以及 CS1602 的课程大作业。在此之外, 随着本课程的进行我对代码进行了多次重构和改进, 以适应本课程的要求和内容。因此, 这一代码的历史较为复杂, 包含了多个阶段的开发和优化。但无论如何, 这一代码在本课程中得到了充分的使用和展示, 是我在本课程学习过程中重要的成果之一。

最后的最后, 我还想说明一点, 尽管使用 sympy, numpy 等成熟科学计算库无疑是更明智的选择, 但是我如今选择从零开始写一整个关于矩阵的库无疑给我带来了更大的提升。感谢本课程让我对线性代数的理解更深入。感谢蒋老师的授课, 感谢助教老师的辛勤付出。

本项目的更多资料, 包括原始文件, 最初实现, 以及 README 等, 可以在我的 GitHub 仓库中找到: https://github.com/JinShuo-Li/Matrix-Library.

2026 年 5 月 7 日

李谨硕